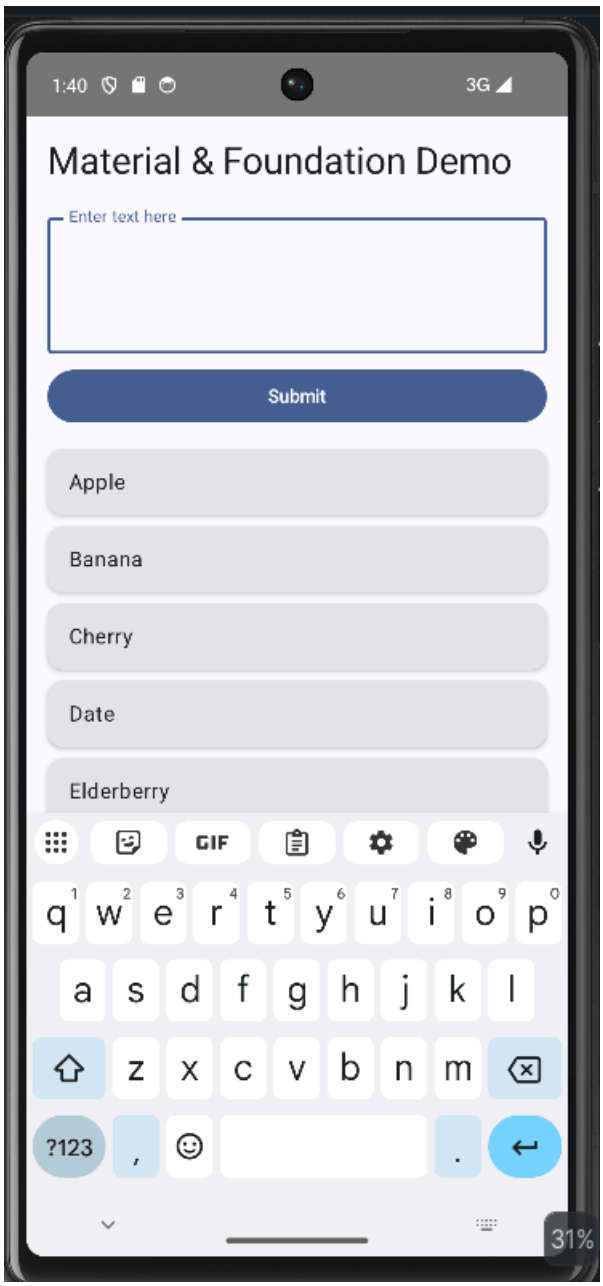
NAME : PIYUSH SHARMA

ROLL NO : 25

REGISTRATION NO. : 12324259

COURSE CODE : CSE 226

CODE



```
package com.example.componentdemo

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.animation.*
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.grid.GridCells
import androidx.compose.foundation.lazy.grid.LazyVerticalGrid
import androidx.compose.foundation.lazy.grid.items
import androidx.compose.foundation.lazy.items
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.automirrored.filled.ArrowBack
import androidx.compose.material.icons.filled.Add
import androidx.compose.material.icons.filled.Remove
import androidx.compose.material.icons.filled.ShoppingCart
import java.util.Locale
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.platform.LocalConfiguration
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextOverflow
import androidx.compose.ui.unit.dp
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.navigation.NavController
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.rememberNavController
import coil.compose.AsyncImage
import kotlinx.coroutines.delay
```

```
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.launch

// --- Data Models ---
data class Product(
    val id: Int,
    val name: String,
    val price: Double,
    val description: String,
    val imageUrl: String
)

data class CartItem(
    val product: Product,
    var quantity: Int
)

// --- ViewModel ---
class ShoppingViewModel : ViewModel() {
    private val _products = MutableStateFlow<List<Product>>(emptyList())
    val products = _products.asStateFlow()

    private val _cart = MutableStateFlow<List<CartItem>>(emptyList())
    val cart = _cart.asStateFlow()

    private val _isLoading = MutableStateFlow(true)
    val isLoading = _isLoading.asStateFlow()

    init {
        loadProducts()
    }

    private fun loadProducts() {
        viewModelScope.launch {
            // Simulate network delay
            delay(1500)
```

```
            _products.value = listOf(
                Product(1, "Premium Headphones", 299.99, "Noise-cancelling over-ear headphones with superior sound quality.", "https://picsum.photos/id/1/300/300"),
                Product(2, "Smart Watch", 199.50, "Track your fitness, heart rate, and notifications in style.", "https://picsum.photos/id/2/300/300"),
                Product(3, "Wireless Mouse", 45.00, "Ergonomic wireless mouse with precision tracking.", "https://picsum.photos/id/3/300/300"),
                Product(4, "Mechanical Keyboard", 129.99, "RGB mechanical keyboard with tactile switches.", "https://picsum.photos/id/4/300/300"),
                Product(5, "Portable Charger", 39.99, "20000mAh power bank for all your devices.", "https://picsum.photos/id/5/300/300"),
                Product(6, "Bluetooth Speaker", 89.95, "Waterproof speaker with 360-degree sound.", "https://picsum.photos/id/6/300/300")
            )
            _isLoading.value = false
        }
    }

    fun addToCart(product: Product) {
        val currentCart = _cart.value.toMutableList()
        val existingItem = currentCart.find { it.product.id == product.id }
        if (existingItem != null) {
            val index = currentCart.indexOf(existingItem)
            currentCart[index] = existingItem.copy(quantity = existingItem.quantity + 1)
        } else {
            currentCart.add(CartItem(product, 1))
        }
        _cart.value = currentCart
    }

    fun removeFromCart(product: Product) {
        val currentCart = _cart.value.toMutableList()
        val existingItem = currentCart.find { it.product.id == product.id }
        if (existingItem != null) {
            if (existingItem.quantity > 1) {
```

```
            val index = currentCart.indexOf(existingItem)
            currentCart[index] = existingItem.copy(quantity =
existingItem.quantity - 1)
        } else {
            currentCart.remove(existingItem)
        }
    }
    _cart.value = currentCart
  }
}

// --- Main Activity ---
class MainActivity : ComponentActivity() {
  override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContent {
      MaterialTheme {
        Surface(color = MaterialTheme.colorScheme.background) {
          ShoppingApp()
        }
      }
    }
  }
}

@Composable
fun ShoppingApp() {
  val navController = rememberNavController()
  val viewModel: ShoppingViewModel = viewModel()
  val products by viewModel.products.collectAsState()

  NavHost(navController = navController, startDestination = "catalogue") {
    composable("catalogue") {
      ProductCatalogueScreen(viewModel, navController)
    }
    composable("productDetail/{productId}") { backStackEntry ->
      val productId =
```

```
backStackEntry.arguments?.getString("productId")?.toIntOrNull()
        val product = remember(products, productId) { products.find { it.id ==
productId } }
        product?.let {
            ProductDetailScreen(it, viewModel, navController)
        }
    }
    composable("cart") {
        CartScreen(viewModel, navController)
    }
  }
}

// --- Screens ---

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ProductCatalogueScreen(viewModel: ShoppingViewModel, navController:
NavController) {
  val products by viewModel.products.collectAsState()
  val isLoading by viewModel.isLoading.collectAsState()
  val cart by viewModel.cart.collectAsState()
  val cartCount = cart.sumOf { it.quantity }

  val configuration = LocalConfiguration.current
  val isTablet = configuration.screenWidthDp >= 600

  Scaffold(
    topBar = {
      TopAppBar(
        title = { Text("Gadget Shop") },
        actions = {
          BadgedBox(
            badge = {
              if (cartCount > 0) {
                Badge { Text(cartCount.toString()) }
              }
```

```
                }
            ) {
                IconButton(onClick = { navController.navigate("cart") }) {
                    Icon(Icons.Default.ShoppingCart, contentDescription = "Cart")
                }
            }
        }
    )
}
) { padding ->
    Box(modifier = Modifier.padding(padding)) {
        if (isLoading) {
            CircularProgressIndicator(modifier = Modifier.align(Alignment.Center))
        } else {
            if (isTablet) {
                LazyVerticalGrid(
                    columns = GridCells.Fixed(3),
                    contentPadding = PaddingValues(16.dp),
                    horizontalArrangement = Arrangement.spacedBy(16.dp),
                    verticalArrangement = Arrangement.spacedBy(16.dp)
                ) {
                    items(products) { product ->
                        ProductItem(product) {
                            navController.navigate("productDetail/${product.id}")
                        }
                    }
                }
            } else {
                LazyColumn(
                    contentPadding = PaddingValues(16.dp),
                    verticalArrangement = Arrangement.spacedBy(16.dp)
                ) {
                    items(products) { product ->
                        ProductItem(product) {
                            navController.navigate("productDetail/${product.id}")
                        }
                    }
```

```
            }
        }
    }
  }
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ProductDetailScreen(product: Product, viewModel: ShoppingViewModel,
navController: NavController) {
  Scaffold(
    topBar = {
      TopAppBar(
        title = { Text("Product Details") },
        navigationIcon = {
          IconButton(onClick = { navController.popBackStack() }) {
            Icon(Icons.AutoMirrored.Filled.ArrowBack, contentDescription =
"Back")
          }
        }
      )
    }
  ) { padding ->
    Column(
      modifier = Modifier
        .padding(padding)
        .fillMaxSize()
        .padding(16.dp)
    ) {
      AsyncImage(
        model = product.imageUrl,
        contentDescription = product.name,
        modifier = Modifier
          .fillMaxWidth()
          .height(300.dp),
        contentScale = ContentScale.Crop
```

```
        )
        Spacer(modifier = Modifier.height(16.dp))
        Text(text = product.name, style =
MaterialTheme.typography.headlineLarge)
        Text(
            text = "$${product.price}",
            style = MaterialTheme.typography.titleLarge,
            color = MaterialTheme.colorScheme.primary
        )
        Spacer(modifier = Modifier.height(8.dp))
        Text(text = product.description, style =
MaterialTheme.typography.bodyLarge)
        Spacer(modifier = Modifier.weight(1f))
        Button(
            onClick = { viewModel.addToCart(product) },
            modifier = Modifier.fillMaxWidth()
        ) {
            Text("Add to Cart")
        }
    }
  }
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun CartScreen(viewModel: ShoppingViewModel, navController: NavController)
{
  val cart by viewModel.cart.collectAsState()

  Scaffold(
    topBar = {
      TopAppBar(
        title = { Text("My Cart") },
        navigationIcon = {
          IconButton(onClick = { navController.popBackStack() }) {
            Icon(Icons.AutoMirrored.Filled.ArrowBack, contentDescription =
"Back")
```

```
                }
            }
        )
    }
) { padding ->
    if (cart.isEmpty()) {
        Box(modifier = Modifier.fillMaxSize(), contentAlignment =
Alignment.Center) {
            Text("Your cart is empty")
        }
    } else {
        LazyColumn(
            modifier = Modifier
                .padding(padding)
                .fillMaxSize(),
            contentPadding = PaddingValues(16.dp),
            verticalArrangement = Arrangement.spacedBy(12.dp)
        ) {
            items(cart, key = { it.product.id }) { cartItem ->
                AnimatedVisibility(
                    visible = true,
                    enter = expandVertically() + fadeIn(),
                    exit = shrinkVertically() + fadeOut()
                ) {
                    CartItemView(cartItem,
                        onIncrease = { viewModel.addToCart(cartItem.product) },
                        onDecrease = { viewModel.removeFromCart(cartItem.product)
}
                    )
                }
            }
            item {
                HorizontalDivider(modifier = Modifier.padding(vertical = 16.dp))
                val totalPrice = cart.sumOf { it.product.price * it.quantity }
                Row(
                    modifier = Modifier.fillMaxWidth(),
                    horizontalArrangement = Arrangement.SpaceBetween
```

```
            ) {
                Text("Total Price:", style = MaterialTheme.typography.titleLarge)
                Text("$${String.format(Locale.getDefault(), "%.2f", totalPrice)}",
style = MaterialTheme.typography.titleLarge, fontWeight = FontWeight.Bold)
            }
            Spacer(modifier = Modifier.height(24.dp))
            Button(onClick = { /* Proceed to checkout */ }, modifier =
Modifier.fillMaxWidth()) {
                Text("Checkout")
            }
        }
    }
  }
}

// --- Reusable UI Components ---

@Composable
fun ProductItem(product: Product, onClick: () -> Unit) {
  Card(
    modifier = Modifier
      .fillMaxWidth()
      .clickable(onClick = onClick),
    elevation = CardDefaults.cardElevation(defaultElevation = 4.dp)
  ) {
    Column {
      AsyncImage(
        model = product.imageUrl,
        contentDescription = product.name,
        modifier = Modifier
          .fillMaxWidth()
          .height(180.dp),
        contentScale = ContentScale.Crop
      )
      Column(modifier = Modifier.padding(12.dp)) {
        Text(
```

```
                text = product.name,
                style = MaterialTheme.typography.titleMedium,
                maxLines = 1,
                overflow = TextOverflow.Ellipsis
            )
            Text(
                text = "$${product.price}",
                style = MaterialTheme.typography.bodyLarge,
                color = MaterialTheme.colorScheme.primary,
                fontWeight = FontWeight.Bold
            )
        }
    }
  }
}

@Composable
fun CartItemView(cartItem: CartItem, onIncrease: () -> Unit, onDecrease: () ->
Unit) {
  Card(
    modifier = Modifier.fillMaxWidth(),
    colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surfaceVariant)
  ) {
    Row(
      modifier = Modifier
        .padding(12.dp)
        .fillMaxWidth(),
      verticalAlignment = Alignment.CenterVertically
    ) {
      AsyncImage(
        model = cartItem.product.imageUrl,
        contentDescription = cartItem.product.name,
        modifier = Modifier
          .size(80.dp),
        contentScale = ContentScale.Crop
      )
```

```
        Spacer(modifier = Modifier.width(16.dp))
        Column(modifier = Modifier.weight(1f)) {
            Text(text = cartItem.product.name, style =
MaterialTheme.typography.titleMedium)
            Text(text = "$${cartItem.product.price}", style =
MaterialTheme.typography.bodyMedium)
        }
        Row(verticalAlignment = Alignment.CenterVertically) {
            IconButton(onClick = onDecrease) {
                Icon(
                    imageVector = Icons.Default.Remove,
                    contentDescription = "Decrease",
                    tint = MaterialTheme.colorScheme.error
                )
            }
            Text(text = cartItem.quantity.toString(), style =
MaterialTheme.typography.bodyLarge)
            IconButton(onClick = onIncrease) {
                Icon(
                    imageVector = Icons.Default.Add,
                    contentDescription = "Increase",
                    tint = MaterialTheme.colorScheme.primary
                )
            }
        }
    }
  }
}
```